Kernels and Random Feature Maps — Project Report

Davide Manfredini (davide.manfredini1@studenti.unimi.it)
Università degli Studi di Milano — *Statistical Methods for Machine Learning* (2025/2026)

1 Theoretical Introduction

1.1 Linear Predictors and Non-Linear Feature Expansion

In binary classification on $\mathcal{X} \subseteq \mathbb{R}^d$ with labels $\mathcal{Y} = \{-1, +1\}$, a linear predictor takes the form

$$h(\mathbf{x}) = \operatorname{sgn}(\mathbf{w}^\top \mathbf{x} + b), \qquad \mathbf{w} \in \mathbb{R}^d,\ b \in \mathbb{R}. \tag{1}$$

Linear models require only $d + 1$ parameters independently of the training sample size m and have $O(d)$ test-time evaluation. However, whenever the true class boundary in $\mathbb{R}^d$ is curved or non-linearly separable, any flat hyperplane suffers from a large *approximation error* (high bias).

To handle non-linear patterns while keeping linear learning algorithms, inputs can be mapped into a higher-dimensional feature space $\mathcal{H}$ through a non-linear transformation $\boldsymbol{\phi} : \mathbb{R}^d \to \mathcal{H}$. A linear hyperplane in $\mathcal{H}$ folds back into a non-linear decision surface in the original space $\mathbb{R}^d$. However, computing $\boldsymbol{\phi}(\mathbf{x})$ explicitly becomes impractical and computational expensive. For example a polynomial expansion of degree n produces $\binom{d+n}{n}$ multinomial features.

1.2 The Kernel Trick and Kernel Bottlenecks

The *Kernel Trick* bypasses explicit feature expansion by exploiting how linear margin algorithms use the training data. For instance, the homogeneous Perceptron training algorithm unrolled over the set S of mistake indices yields $\mathbf{w} = \sum_{s \in S} y_s \mathbf{x}_s$, so prediction rewrites in terms of scalar inner products:

$$h(\mathbf{x}) = \operatorname{sgn}\big(\mathbf{w}^\top \mathbf{x}\big) = \operatorname{sgn}\left(\left(\sum_{s \in S} y_s \mathbf{x}_s\right)^\top \mathbf{x}\right) = \operatorname{sgn}\left(\sum_{s \in S} y_s \, (\mathbf{x}_s^\top \mathbf{x})\right). \tag{2}$$

Applying $\boldsymbol{\phi} : \mathbb{R}^d \to \mathcal{H}$ replaces $(\mathbf{x}_s^\top \mathbf{x})$ with $\langle \boldsymbol{\phi}(\mathbf{x}_s), \boldsymbol{\phi}(\mathbf{x}) \rangle_{\mathcal{H}}$, eliminating the explicit weight vector in $\mathcal{H}$. A positive semi-definite *kernel function* $K : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ evaluates this inner product directly from the input coordinates as $K(\mathbf{x}, \mathbf{x}') = \langle \boldsymbol{\phi}_K(\mathbf{x}), \boldsymbol{\phi}_K(\mathbf{x}') \rangle_{\mathcal{H}_K}$ in $O(d)$ time, where $\mathcal{H}_K$ is the Reproducing Kernel Hilbert Space generated by the kernel function. Geometrically $K(\mathbf{x}, \mathbf{x}')$ acts as a non-linear *similarity measure* in $\mathbb{R}^d$.

In this project, two shift-invariant kernels (kernels depending only on the difference $\mathbf{x} - \mathbf{x}'$) are implemented for the experiments:

$$K_{\text{RBF}}(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \, \|\mathbf{x} - \mathbf{x}'\|_2^2\right), \qquad K_{\text{Lap}}(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \, \|\mathbf{x} - \mathbf{x}'\|_1\right), \tag{3}$$

where the bandwidth $\gamma > 0$ acts as an inverse spatial scale controlling how rapidly similarity decays with distance (note that γ in Equation (3) follows the convention of Rahimi and Recht).

- The **Gaussian RBF kernel** measures smooth ℓ_2 similarity; its Taylor expansion shows that $\mathcal{H}_K$ is infinite-dimensional and contains polynomial interactions of all degrees.
- The **Laplacian kernel** relies instead on the ℓ_1 (Manhattan) distance: it exhibits a sharp non-differentiable peak at $\mathbf{x} = \mathbf{x}'$ and heavier tails.

Both kernels are *universal* on compact subsets of $\mathbb{R}^d$: their RKHS $\mathcal{H}_K$ is dense in the space of continuous functions, so their hypothesis approximation error with respect to any continuous target boundary is zero. Consequently, regularized classifiers such as Soft-Margin SVMs trained with these kernels are *statistically consistent*, meaning that as $m \to \infty$ their expected test error converges to the theoretical minimum Bayes risk.

Despite these statistical advantages, the implicit expansion ties computation directly to the sample size m, introducing two major practical bottlenecks:

1. **Storage and training bottleneck**: Evaluating and storing the full Gram matrix $\mathbf{K} \in \mathbb{R}^{m \times m}$ with entries $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ requires $O(m^2)$ memory and $O(m^2 d)$ operations.
2. **Inference-time bottleneck**: kernel predictor is non-parametric and must store all active support vectors $\mathcal{S} \subseteq \{1, \ldots, m\}$. Since $|\mathcal{S}|$ typically grows with m, predicting the label of a single test point requires $O(md)$ operations.

1.3 Explicit Random Feature Maps and Project Overview

In their 2007 paper *Random Features for Large-Scale Kernel Machines*, Ali Rahimi and Benjamin Recht proposed an alternative approach: instead of relying on the implicit Kernel Trick, inputs are mapped explicitly into a randomized, finite-dimensional Euclidean space through $\mathbf{z} : \mathbb{R}^d \to \mathbb{R}^D$ so that standard inner products approximate the kernel:

$$K(\mathbf{x}, \mathbf{x}') \approx \mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}'). \tag{4}$$

Once the data is transformed into $\mathbf{z}(\mathbf{x}) \in \mathbb{R}^D$, a standard linear model $f_D(\mathbf{x}) = \mathbf{w}^\top \mathbf{z}(\mathbf{x})$ is trained directly in the primal space $\mathbb{R}^D$. When $D \ll m$, explicit random features combine the non-linear expressiveness of kernels with the computational efficiency of linear predictors. At the same time, this approach introduces clear trade-offs: because $\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}')$ is a randomized Monte Carlo estimate, it incurs a representation error that can hurt accuracy if D is too small relative to the ambient dimension d or if the bandwidth γ is very large, and the computational advantage disappears whenever D approaches or exceeds m.

In this project, both families of randomized maps introduced by Rahimi and Recht—**Random Fourier Features** and grid-based **Random Binning Features** for the Laplacian kernel— are analyzed, tested and trained using **Primal and Kernelized Pegasos SVM**.

2 Mathematical Foundations and Implemented Algorithms

2.1 Reproducing Kernel Hilbert Spaces and Shift-Invariant Kernels

A symmetric function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a *positive semi-definite (PSD) kernel* if every finite Gram matrix $\mathbf{K} \in \mathbb{R}^{m \times m}$ ($K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$) satisfies $\mathbf{c}^\top \mathbf{K} \mathbf{c} \geq 0$ for all $\mathbf{c} \in \mathbb{R}^m$. By the Moore-Aronszajn theorem, K uniquely defines a *Reproducing Kernel Hilbert Space* $\mathcal{H}_K$ where each input $\mathbf{x}$ maps to $\boldsymbol{\phi}_K(\mathbf{x}) \equiv K(\mathbf{x}, \cdot)$ and functions $g(\cdot) = \sum_{i=1}^m \beta_i K(\mathbf{x}_i, \cdot)$ satisfy the *Reproducing Property*:

$$g(\mathbf{x}) = \langle g, K(\mathbf{x}, \cdot) \rangle_{\mathcal{H}_K}, \qquad \|g\|^2_{\mathcal{H}_K} = \langle g, g \rangle_{\mathcal{H}_K} = \sum_{i,j=1}^m \beta_i \beta_j K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\beta}^\top \mathbf{K} \boldsymbol{\beta}. \tag{5}$$

Geometrically, $g \in \mathcal{H}_K$ acts as a linear separator on the lifted points $\boldsymbol{\phi}_K(\mathbf{x})$, while $\|g\|_{\mathcal{H}_K}$ bounds the maximum slope of g in $\mathbb{R}^d$ via Cauchy-Schwarz, so penalizing $\frac{\lambda}{2} \|g\|^2_{\mathcal{H}_K}$ suppresses sharp oscillations. For both shift-invariant kernels in Equation (3), $K(\mathbf{x}, \mathbf{x}) = \|\boldsymbol{\phi}_K(\mathbf{x})\|^2_{\mathcal{H}_K} = 1$, placing every mapped input on the unit sphere of $\mathcal{H}_K$. Moreover, the bandwidth γ controls spatial locality analogously to k-Nearest Neighbors: as $\gamma \to 0$, $K(\mathbf{x}_i, \mathbf{x}) \to 1$ globally so predictions collapse to a uniform average (underfitting); as $\gamma \to \infty$, $\mathbf{K} \to \mathbf{I}_m$ and kernel bumps shrink into narrow spikes dominated by the closest training point (1-NN memorization, risking overfitting).

2.2 Soft-Margin SVM and Pegasos Optimization

To separate overlapping classes, the λ-strongly convex Soft-Margin SVM objective ($\lambda > 0$) is minimized, combining the hinge loss $[1 - u]_+ \equiv \max(0, 1 - u)$ with L_2 regularization:

$$F(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m \left[1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right]_+ + \frac{\lambda}{2} \left(\|\mathbf{w}\|_2^2 + \mathbb{I}_{\text{int}} b^2 \right), \tag{6}$$

where $\mathbb{I}_{\text{int}} \in \{0, 1\}$ toggles a regularized intercept (`fit_intercept`). Geometrically, $f(\mathbf{x}) = 0$ is the decision boundary and $f(\mathbf{x}) = \pm 1$ are the *margin gutters* (width $2/\|\mathbf{w}\|_2$), with points satisfying $y_i f(\mathbf{x}_i) < 1$ acting as active support vectors. Then it is set `fit_intercept=True` for the raw linear baseline in $\mathbb{R}^d$, and `fit_intercept=False` ($f_D(\mathbf{x}) = \mathbf{w}^\top \mathbf{z}(\mathbf{x})$) on random features $\mathbf{z}(\mathbf{x}) \in \mathbb{R}^D$ to match the homogeneous RKHS objective of our exact kernel solver.

Primal Pegasos (`PrimalPegasosSVM`). Pegasos minimizes Equation (6) via stochastic subgradient descent from $\mathbf{w}_1 = \mathbf{0}$ ($b_1 = 0$) with step size $\eta_t = \frac{1}{\lambda t}$. At step $t \in \{1, \ldots, T\}$, sampling $i_t \sim \text{Uniform}\{1, \ldots, m\}$ and setting $\mathbf{v}_t = y_{i_t} \mathbf{x}_{i_t} \mathbb{I}\{y_{i_t} f_t(\mathbf{x}_{i_t}) < 1\}$ gives the update and its unrolled sum:

$$\mathbf{w}_{t+1} = \left(1 - \frac{1}{t}\right) \mathbf{w}_t + \frac{1}{\lambda t} \mathbf{v}_t = \frac{1}{\lambda t} \sum_{s=1}^t \mathbf{v}_s \tag{7}$$

At each step, $(1 - 1/t)$ shrinks $\|\mathbf{w}_t\|_2$ to widen the margin while margin violations add $\frac{y_{i_t}}{\lambda t} \mathbf{x}_{i_t}$ to adjust the separator.

Kernelized Pegasos (`KernelPegasosSVM`). Because the unrolled sum in Equation (7) expresses $\mathbf{w}_t$ solely as a linear combination of past margin-violating training points, the Kernel Trick ($\mathbf{x}_i^\top \mathbf{x} \mapsto K(\mathbf{x}_i, \mathbf{x})$) can be applied without computing explicit weights in $\mathcal{H}_K$. Grouping past updates by sample index i via an integer counter $c_i^{(t-1)}$ (counting how many times $\mathbf{x}_i$ violated the margin before step t), the predictor for $t \geq 2$ ($g_1 \equiv 0$) becomes

$$g_t(\mathbf{x}) = \frac{1}{\lambda(t-1)} \sum_{i=1}^{m} c_i^{(t-1)} y_i K(\mathbf{x}_i, \mathbf{x}). \tag{8}$$

At step t, if the sampled point $\mathbf{x}_{i_t}$ satisfies $y_{i_t} g_t(\mathbf{x}_{i_t}) < 1$, its counter is updated as $c_{i_t} \leftarrow c_{i_t} + 1$. After T steps, only the active support vectors $\mathcal{S} = \{i : c_i^{(T)} > 0\}$ have non-zero coefficients β_i and must be stored to predict $g(\mathbf{x}) = \sum_{i \in \mathcal{S}} \beta_i K(\mathbf{x}_i, \mathbf{x})$.

2.3 Bochner's Theorem, Random Fourier Features, and Uniform Convergence

To bypass the $O(m^2)$ Gram matrix and $O(md)$ per-step cost of Equation (8), Rahimi and Recht invoke **Bochner's Theorem**: a continuous shift-invariant function $K(\mathbf{x}, \mathbf{x}') = k(\boldsymbol{\Delta})$ ($\boldsymbol{\Delta} = \mathbf{x} - \mathbf{x}' \in \mathbb{R}^d$) is positive definite if and only if $k(\boldsymbol{\Delta})$ is the Fourier transform of a non-negative measure. Since $k(\mathbf{0}) = 1$, the spectral density $p(\boldsymbol{\omega}) \geq 0$ integrates to $\int_{\mathbb{R}^d} p(\boldsymbol{\omega})\, d\boldsymbol{\omega} = k(\mathbf{0}) = 1$ and forms a valid *probability density function* over frequencies $\boldsymbol{\omega} \in \mathbb{R}^d$. Because $k(\boldsymbol{\Delta})$ and $p(\boldsymbol{\omega})$ are real and even, the imaginary sine integral vanishes:

$$K(\mathbf{x}, \mathbf{x}') = \int_{\mathbb{R}^d} p(\boldsymbol{\omega}) e^{i\boldsymbol{\omega}^\top(\mathbf{x}-\mathbf{x}')}\, d\boldsymbol{\omega} = \mathbb{E}_{\boldsymbol{\omega} \sim p(\boldsymbol{\omega})}\left[\cos\left(\boldsymbol{\omega}^\top(\mathbf{x} - \mathbf{x}')\right)\right]. \tag{9}$$

Taking the inverse Fourier transform of Equation (3) yields the two spectral distributions sampled in our code: for **Gaussian RBF**, $p(\boldsymbol{\omega}) = (4\pi\gamma)^{-d/2} \exp(-\|\boldsymbol{\omega}\|_2^2/(4\gamma)) = \mathcal{N}(\mathbf{0}, 2\gamma \mathbf{I}_d)$ ($\omega_{s,j} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 2\gamma)$); for **Laplacian**, $p(\boldsymbol{\omega}) = \prod_{s=1}^{d} \frac{1}{\pi\gamma(1+(\omega_s/\gamma)^2)} = \prod_{s=1}^{d} \text{Cauchy}(0, \gamma)$, sampled via inverse CDF $\omega_{s,j} = \gamma \tan(U_{s,j})$ with $U_{s,j} \sim \text{Uniform}(-\pi/2, \pi/2)$.

Geometric Meaning and Unification with Primal Pegasos. Each vector $\boldsymbol{\omega}_j \in \mathbb{R}^d$ specifies a wave direction and spatial oscillation frequency $\|\boldsymbol{\omega}_j\|_2$, projecting $\mathbf{x}$ onto a sinusoidal plane wave across $\mathbb{R}^d$. Averaging D waves in $\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}')$ produces *constructive interference* at $\boldsymbol{\Delta} = \mathbf{0}$ and *destructive cancellation* at large $\|\boldsymbol{\Delta}\|_2$, reconstructing the localized similarity bump. By the Fourier uncertainty principle, a narrow spatial spike (large γ) requires a wide frequency spectrum ($\text{Var}(\omega_s) = 2\gamma$). Substituting $K(\mathbf{x}_{i_s}, \mathbf{x}) \approx \mathbf{z}(\mathbf{x}_{i_s})^\top \mathbf{z}(\mathbf{x})$ into Equation (8) factors out $\mathbf{z}(\mathbf{x})$:

$$g_{t+1}(\mathbf{x}) \approx \frac{1}{\lambda t} \sum_{s=1}^{t} y_{i_s} \mathbb{I}\{\dots\} \mathbf{z}(\mathbf{x}_{i_s})^\top \mathbf{z}(\mathbf{x}) = \left(\frac{1}{\lambda t} \sum_{s=1}^{t} y_{i_s} \mathbb{I}\{\dots\} \mathbf{z}(\mathbf{x}_{i_s}) \right)^\top \mathbf{z}(\mathbf{x}) = \mathbf{w}_{t+1}^\top \mathbf{z}(\mathbf{x}), \tag{10}$$

where $\mathbf{w}_{t+1} \in \mathbb{R}^D$ is precisely the Primal Pegasos iterate Equation (7) on $\mathbf{z}(\mathbf{x})$. Thus, Primal Pegasos in $\mathbb{R}^D$ directly approximates Kernel Pegasos in $\mathcal{H}_K$, collapsing the growing support-vector sum into a single vector $\mathbf{w} \in \mathbb{R}^D$.

Pointwise and Uniform Convergence (Claim 1). For a fixed pair $(\mathbf{x}, \mathbf{x}')$, single-feature products lie in $[-2, 2]$, so Hoeffding's inequality bounds $\mathbb{P}[|\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}') - K(\mathbf{x}, \mathbf{x}')| \geq \epsilon] \leq 2\exp(-D\epsilon^2/4)$, giving the Monte Carlo rate $O(D^{-1/2})$ (log-log slope -0.5). Across an entire compact domain $\mathcal{M} \subset \mathbb{R}^d$ of diameter $\text{diam}(\mathcal{M})$, Claim 1 of Rahimi and Recht combines an r-net covering of $\mathcal{M}_\Delta$ with Markov's bound on the Lipschitz slope via the spectral second moment $\sigma_p^2 \equiv \mathbb{E}_p[\|\boldsymbol{\omega}\|_2^2] = -\text{Tr}(\nabla^2 k(\mathbf{0}))$:

$$\mathbb{P}\left[\sup_{\mathbf{x}, \mathbf{x}' \in \mathcal{M}} \left| \mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}') - K(\mathbf{x}, \mathbf{x}') \right| \geq \epsilon \right] \leq 2^8 \left(\frac{\sigma_p \,\text{diam}(\mathcal{M})}{\epsilon} \right)^2 \exp\left(-\frac{D\epsilon^2}{4(d+2)} \right), \tag{11}$$

yielding uniform error $\leq \epsilon$ when $D = \Omega\left(\frac{d}{\epsilon^2} \log \frac{\sigma_p \,\text{diam}(\mathcal{M})}{\epsilon} \right)$. Geometrically, σ_p^2 measures kernel curvature at $\mathbf{0}$: for RBF, $\sigma_p^2 = 2d\gamma$, explaining why larger d or γ demands more features D to avoid high-frequency aliasing; for the non-differentiable Laplacian cusp, $\sigma_p^2 = +\infty$ (Cauchy frequencies have infinite variance).

2.4 Implemented Random Feature Map Constructions

Following Sections 3 and 4 of Rahimi and Recht, `src/random_features.py` implements the paper's two core randomized algorithms, together with two variance-reduction variants of Algorithm 1 evaluated in Extension 2:

- **Algorithm 1 — Random Fourier Features (`RFFCosinePhase`)**: To approximate a shift-invariant kernel $K(\mathbf{x}, \mathbf{x}') = k(\mathbf{x} - \mathbf{x}')$, the algorithm draws D i.i.d. direction vectors $\boldsymbol{\omega}_1, \ldots, \boldsymbol{\omega}_D \in \mathbb{R}^d$ from its Fourier transform $p(\boldsymbol{\omega})$ (Gaussian for RBF, Cauchy for Laplacian) and D uniform phase shifts $b_1, \ldots, b_D \sim \text{Uniform}[0, 2\pi]$. Each input $\mathbf{x}$ is projected onto these random lines, shifted by b_j, and passed through a sinusoid:

$$\mathbf{z}(\mathbf{x}) = \sqrt{\frac{2}{D}} \big[\cos(\boldsymbol{\omega}_1^\top \mathbf{x} + b_1), \ldots, \cos(\boldsymbol{\omega}_D^\top \mathbf{x} + b_D) \big]^\top \in \mathbb{R}^D. \tag{12}$$

  By the sum-of-angles identity, each component product is an unbiased estimate of $\mathbb{E}[\cos(\boldsymbol{\omega}^\top(\mathbf{x} - \mathbf{x}'))] = K(\mathbf{x}, \mathbf{x}')$, and averaging D independent components lowers the estimation variance. In Extension 2, are also tested two variants of this map: **Paired Cosine-Sine (`RFFCosSinPair`)**, which directly expands the real part of $e^{i\boldsymbol{\omega}^\top(\mathbf{x}-\mathbf{x}')}$ in Equation (9) into $[\cos(\boldsymbol{\omega}_j^\top \mathbf{x}), \sin(\boldsymbol{\omega}_j^\top \mathbf{x})]$ pairs across $D/2$ frequencies without phase shifts b_j; and **Orthogonal Random Features (`OrthogonalRandomFeatures`)**, an RBF variant that draws orthogonal frequency directions within $d \times d$ blocks via QR decomposition (rescaled to keep the Gaussian marginal $p(\boldsymbol{\omega})$) so directions spread more evenly across $\mathbb{R}^d$.
- **Algorithm 2 — Random Binning Features (`RandomBinningFeatures`)**: Rather than using smooth global sinusoids, Random Binning partitions the input space with P randomly shifted rectilinear grids at random resolutions, making it well-suited for ℓ_1-based kernels such as the Laplacian kernel. Along a single coordinate axis, a grid of pitch $\delta > 0$ shifted uniformly by $u \sim \text{Uniform}[0, \delta]$ assigns each coordinate x to bin $\lfloor (x-u)/\delta \rfloor$, so two points x and x' fall in the same bin with probability $\max(0, 1 - |x - x'|/\delta)$ (the 1D "hat" kernel). When each 1D kernel factor $k_s(\Delta)$ is convex for $\Delta \geq 0$, sampling the pitch from $p(\delta) = \delta \ddot{k}_s(\delta)$—a Gamma$(2, 1/\gamma)$ density for the Laplacian kernel, whereas the non-convex Gaussian kernel yields no valid probability distribution—averages the hat kernel into $k_s(|x_s - x'_s|)$. Applying this binning independently across all d axes multiplies the collision probabilities into $K_{\text{Lap}}(\mathbf{x}, \mathbf{x}')$. Each grid encodes the cell containing $\mathbf{x}$ as a binary indicator vector (discarding unoccupied bins to prevent overflow when d is large), and concatenating P independent grids scaled by $1/\sqrt{P}$ yields an inner product $\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}')$ proportional to how often $\mathbf{x}$ and $\mathbf{x}'$ land in the same bin.

3 Experimental Setup and Datasets

3.1 Datasets Construction

All experiments are executed side by side on two complementary binary classification benchmarks ($\mathcal{Y} = \{-1, +1\}$) implemented in `src/datasets.py`: a bi-dimensional synthetic problem with an analytical ground-truth distribution, and a higher-dimensional real-world visual recognition task ($d = 64$) that directly probes the dimensional dependence $D = \Omega(d\epsilon^{-2} \log(\sigma_p \operatorname{diam}(\mathcal{M})/\epsilon))$ of Claim 1 (Equation (11)).

1. Synthetic Noisy Concentric Rings. Data points $\mathbf{x}_i \in \mathbb{R}^2$ are sampled uniformly from the compact box $\mathcal{M} = [-1.8, 1.8]^2$ of diameter $\operatorname{diam}(\mathcal{M}) = 3.6\sqrt{2} \approx 5.091$. The optimal circle latent function $f^*(\mathbf{x}) = \|\mathbf{x}\|_2^2 - r_0^2$ is defined choosing radius $r_0 = 1.436$ so that the inner disk area $\pi r_0^2 \approx 6.48$ equals half the box area $3.6^2 = 12.96$, naturally producing balanced classes (51.67% positive). Labels are generated by adding zero-mean Gaussian noise $\varepsilon_i \sim \mathcal{N}(0, \sigma_{\text{noise}}^2)$ ($\sigma_{\text{noise}} = 0.35$) directly to the latent score:

$$y_i = \operatorname{sgn}\big(\|\mathbf{x}_i\|_2^2 - r_0^2 + \varepsilon_i \big) \in \{-1, +1\}. \tag{13}$$

Because ε_i perturbs the latent threshold rather than shifting coordinates before labeling, Equation (13) produces genuine class infiltration across the boundary, which is essential to expose high-γ overfitting. Since the class-conditional probability is $\eta(\mathbf{x}) = \mathbb{P}(Y = +1 \mid \mathbf{X} = \mathbf{x}) = \Phi(f^*(\mathbf{x})/\sigma_{\text{noise}})$ (where Φ is the standard normal CDF), the exact Bayes optimal boundary $\eta(\mathbf{x}) = 1/2$ is the circle $\|\mathbf{x}\|_2 = r_0$, guaranteeing that any flat linear classifier in $\mathbb{R}^2$ suffers severe approximation bias. It is possible to compute the asymptotic expected Bayes risk ($m \to \infty$) $\mathcal{R}^* = \mathbb{E}_{\mathbf{X}}[\Phi(-|f^*(\mathbf{X})|/\sigma_{\text{noise}})] = 6.61\%$ (via `scipy.special.ndtr`).

2. Real-World UCI Handwritten Digits Even-vs-Odd ($m = 1{,}797$, $d = 64$). Then it is used the UCI Optical Recognition of Handwritten Digits dataset (`sklearn.datasets.load_digits`), consisting of 1,797 grayscale 8×8 images flattened into $\mathbb{R}^{64}$ with integer intensities in $[0, 16]$, and map digits by parity: even digits $\{0, 2, 4, 6, 8\} \mapsto -1$ (891 samples, 49.58%) versus odd digits $\{1, 3, 5, 7, 9\} \mapsto +1$ (906 samples, 50.42%). The arbitrary task in this case is to separate even digits from odd digits.

3.2 Preprocessing and Hyperparameter Selection

To prevent any statistical leakage from the test distribution, the pipeline executes preprocessing and model selection following this chronological order:

1. **Train-Test Split**: First, `prepare_dataset_zero_leakage` independently permutes negative and positive indices and holds out 30% of each class as $(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}})$ (1,050/450 on `Synthetic_d2`; 1,258/539 on `Digits_d64`). The test split is never accessed during preprocessing or tuning.
2. **Standardization**: On `Synthetic_d2` (`standardize=False`), raw coordinates are kept untouched because both axes share the same uniform scale on $[-1.8, 1.8]$ and keeping raw units preserves the exact circle radius $r_0 = 1.436$. On `Digits_d64` (`standardize=True`), pixel variances differ sharply between central strokes and borders. Per-feature mean μ_k and standard deviation σ_k are therefore estimated exclusively on $\mathbf{X}_{\text{train}}$ and the normalization is applyed on $\mathbf{X}_{\text{train}}$ and $\mathbf{X}_{\text{test}}$ via $(x_{i,k} - \mu_k)/\sigma_k$.
3. **5-Fold Cross-Validation hyperparameter selection on** $(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$: Using `KernelPegasosSVM`, hyperparameters are selected via the minimum mean validation 0-1 error rule (`selection_rule="min"`). First, $(\gamma^*_{\text{RBF}}, \lambda^*)$ are selected jointly over the 2D candidate grids; next, fixing $\lambda = \lambda^*$, γ^*_{Lap} is tuned also for the Laplacian kernel. This yields $(\gamma^*_{\text{RBF}}, \gamma^*_{\text{Lap}}, \lambda^*) = (1.0, 1.0, 10^{-3})$ on `Synthetic_d2` and $(0.015, 0.02, 2 \cdot 10^{-4})$ on `Digits_d64`: because $\|\mathbf{x} - \mathbf{x}'\|_2^2 = \sum_{k=1}^{d} (x_k - x'_k)^2$ sums d coordinate differences, distances between points are much larger in $d = 64$ than in $d = 2$, so a smaller γ is needed to prevent $\exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|_2^2)$ from collapsing to zero. All linear baselines and random feature models subsequently keep (γ^*, λ^*) fixed so that D (or γ in Extension 1) is the sole varying factor.

3.3 Evaluation Metrics and Averaging

Across our experiments, predictive performance is measured via the empirical 0-1 error rate $\widehat{\mathcal{R}} = \frac{1}{m_{\text{eval}}} \sum_{i=1}^{m_{\text{eval}}} \mathbb{I}\{\hat{y}_i \neq y_i\}$, while kernel representation fidelity between the exact training Gram matrix $\mathbf{K} = K(\mathbf{X}_{\text{train}}, \mathbf{X}_{\text{train}})$ and its random feature approximation $\widetilde{\mathbf{K}} = \mathbf{Z}_{\text{train}} \mathbf{Z}_{\text{train}}^\top$ is quantified in `src/kernels.py` using both the *Relative Frobenius Error* and the *Supremum (Paper's Claim 1) Error*:

$$\mathcal{E}_{\text{Frob}}(D) = \frac{\left\| \mathbf{K} - \widetilde{\mathbf{K}} \right\|_{\text{F}}}{\|\mathbf{K}\|_{\text{F}}}, \qquad \mathcal{E}_{\infty}(D) = \max_{1 \leq i, j \leq m_{\text{train}}} \left| K_{ij} - \widetilde{K}_{ij} \right|, \tag{14}$$

estimating empirical convergence exponents via ordinary least-squares regression of $\log \mathcal{E}$ against $\log D$. Because both random feature draws $(\mathbf{W}, \mathbf{b})$ and Pegasos sampling (i_t) are stochastic, every randomized configuration in Experiments 1, 3, and 4 is repeated across $S = 10$ independent deterministic seed offsets. For wall-clock runtime benchmarks (Extension 3), results are averaged across $S = 3$ seeds at $T = 10{,}000$ iterations using `time.perf_counter()`.

4 Core Experiments

4.1 Core Experiment 1: Kernel Matrix and Classification Convergence vs. Feature Dimensions D

First it is evaluated how accurately `RFFCosinePhase` (Section 2.4) approximates the exact RBF kernel and its downstream SVM classifier across $D \in \{5, 10, 25, 50, 100, 250, 500, 1{,}000, 2{,}000\}$. From Section 2.3, Monte Carlo averaging predicts $O(D^{-1/2})$ decay (log-log slope -0.50) for both $\mathcal{E}_{\text{Frob}}(D)$ and $\mathcal{E}_{\infty}(D)$ (Equation (14)), while Claim 1 (Equation (11)) requires $D = \Omega(d\epsilon^{-2} \log(\sigma_p \operatorname{diam}(\mathcal{M})/\epsilon))$, predicting a much larger feature budget on `Digits_d64` ($d = 64$) than on `Synthetic_d2` ($d = 2$).

In Figure 1A, log-log regression confirms the theoretical -0.50 exponent on both datasets, with empirical slopes of -0.494 and -0.495 for $\mathcal{E}_{\text{Frob}}$ and -0.492 and -0.486 for $\mathcal{E}_{\infty}$. Moreover, $\mathcal{E}_{\infty}$ lies strictly above $\mathcal{E}_{\text{Frob}}$ because it takes the worst-case maximum over all m^2_{train} pairs rather than a root-mean-square average, and their

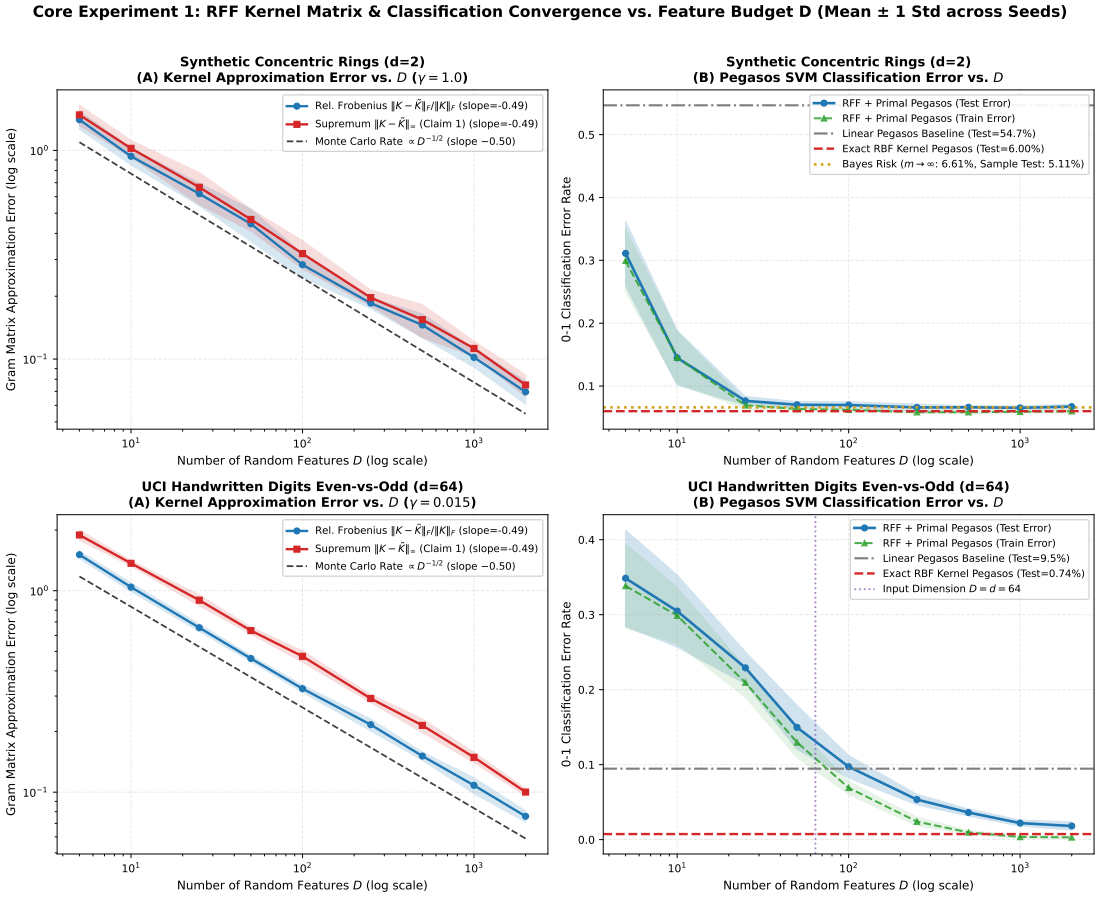


Figure 1: Core Experiment 1 ($S = 10$ seeds, mean ±1 std). **(A)** Log-log Gram approximation errors $\mathcal{E}_{\text{Frob}}(D)$ and $\mathcal{E}_{\infty}(D)$ vs. $D^{-1/2}$. **(B)** Train/test 0-1 error of `RFF + PrimalPegasosSVM` against `Linear` and `Exact RBF` baselines.

vertical gap widens in $d = 64$ due to higher-dimensional supremum fluctuations and sparser off-diagonal kernel entries. In Figure 1B, comparing RFF against the raw `Linear` baseline (54.67% test error on radially symmetric rings; 9.46% on digits) and `Exact RBF` (6.00% and 0.74%) highlights both sides of the approximation:

- **Fast low-dimensional decision convergence** ($d = 2$): On `Synthetic_d2`, test error drops much faster than Gram error, reaching 7.02% already at $D = 50$ (despite $\mathcal{E}_{\text{Frob}} = 0.445$) and 6.53%–6.73% for $D \geq 250$, since classification only requires accurate score signs near the boundary.
- **Dimensional bottleneck** ($D < d$) **and slow tail** ($d = 64$): On `Digits_d64`, whenever $D < d = 64$, RFF (34.86% at $D = 5$; 14.97% at $D = 50$) performs *worse* than raw linear Pegasos (9.46%) because projecting $\mathbb{R}^{64}$ onto $D < 64$ random directions discards informative subspaces (matching linear only near $D = 100$). Furthermore, even at $D = 2{,}000 > m_{\text{train}} = 1{,}258$—where RFF loses any memory or speed benefit—test error (1.82%) still trails Exact RBF (0.74%).

4.2 Core Experiment 2: 2D Geometric Progression of Decision Boundaries

To visualize how random plane waves build non-linear boundaries, a single realization is evaluated on `Synthetic_d2` over a 200×200 grid in $[-2.0, 2.0]^2$ for `Linear`, `RFF + Linear` at $D \in \{5, 15, 50, 500\}$, and `Exact RBF`, measuring both 0-1 errors and the grid score discrepancy $\text{RMSE}_{\text{grid}}$ relative to $f_{\text{exact}}(\mathbf{x})$. As D grows, wave superposition should transition from directional stripes to isotropic radial contours matching $\mathcal{H}_K$.

In Figure 2, the linear baseline (43.78% test) can only place a flat chord across one corner, with conflicting radial gradients shrinking $\|\mathbf{w}\|_2$ so that $|f(\mathbf{x})| < 1$ everywhere (no ±1 gutters appear). At $D = 5$, five cosines produce open parallel stripes (36.44%). By $D = 15$, wave interference closes a loop around the origin (11.33%), but angular under-sampling leaves a distorted contour and a spurious negative pocket in the bottom-left corner. At $D = 50$ (6.89%) and $D = 500$ (6.44%, $\text{RMSE}_{\text{grid}} = 0.180$), ripples vanish and both $f(\mathbf{x}) = 0$ and $f(\mathbf{x}) = \pm 1$ match Exact RBF (5.78%), smoothly ignoring infiltrated noise points.

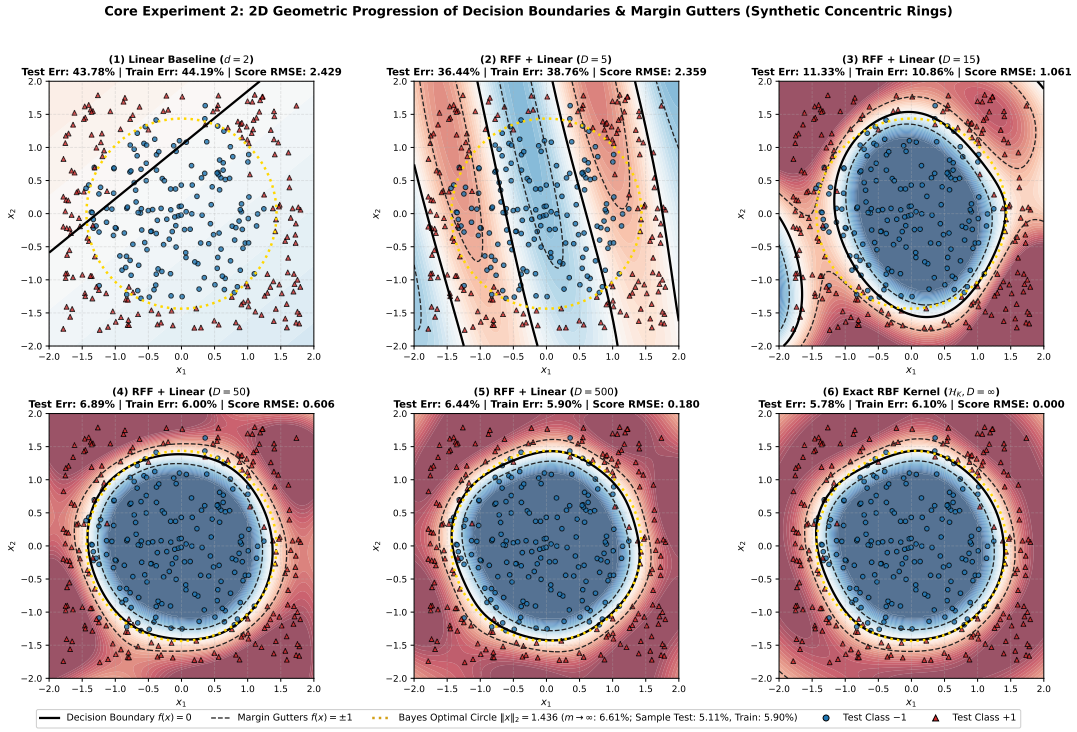


Figure 2: Core Experiment 2 on `Synthetic_d2` (`seed=42`): decision boundary $f(\mathbf{x}) = 0$ (solid), margin gutters $f(\mathbf{x}) = \pm 1$ (dashed), and Bayes circle $\|\mathbf{x}\|_2 = 1.436$ (yellow) on held-out test data.

5 Experimental Extensions

5.1 Extension 1: Effect of Kernel Bandwidth γ on Exact Kernels vs. Random Features

The third experiment compares `Exact RBF` against `RFFCosinePhase` at fixed budgets $D \in \{25, 100, 500\}$, fixing λ^* and varying $\gamma \in [10^{-3}, 10^3]$ on `Synthetic_d2` and $\gamma \in [10^{-4}, 1.0]$ on `Digits_d64`. While γ governs the standard bias–variance trade-off for Exact RBF ($\gamma \to 0$ global averaging vs. $\gamma \to \infty$ 1-NN memorization), paper's Claim 1 (Equation (11)) predicts an extra *spectral aliasing* failure for RFF: since $\sigma_p^2 = 2d\gamma$, large γ samples high-frequency waves that fail to cancel at finite D. Specifically, as $\mathbf{K}_\gamma \to \mathbf{I}_{m_{\text{train}}}$, off-diagonal entries of $\widetilde{\mathbf{K}}_\gamma$ retain variance $1/D$ and diagonal entries $1/(2D)$ by Werner's identity, so $\mathcal{E}_{\text{Frob}}$ saturates at

$$\lim_{\gamma \to \infty} \mathcal{E}_{\text{Frob}}(\gamma, D) = \sqrt{\frac{m_{\text{train}} - 0.5}{D}}. \tag{15}$$

Figure 3A confirms the U-shaped curve of Exact RBF, reaching minima of 4.22% ($\gamma = 10$) on `Synthetic_d2` and 0.56% ($\gamma = 0.03$) on `Digits_d64` and degrading only mildly at large γ (8.00% and 1.48%). Conversely, once $\sigma_p^2 = 2d\gamma$ exceeds the resolution of D, high-frequency aliasing destroys RFF locality and pushes test error back to chance ($\sim$ 40%–51%). Simultaneously, Figure 3B matches Equation (15) closely (e.g., 7.09, 3.55, 1.59 on `Digits_d64` at $\gamma = 1.0$). Positively, near and below γ^*, $D = 500$ tracks Exact RBF accurately, and increasing D shifts aliasing to higher γ.

5.2 Extension 2: Comparison of Random Feature Map Constructions

The fourth experiment compare all five constructions of random feature maps from Section 2.4: Phase Cosine, Cos-Sin Pair, and ORF ($D \in [10, 1{,}000]$) for the RBF kernel and Random Binning versus Cauchy RFF ($D = P \in [1, 200]$) for the Laplacian kernel, at fixed $(\gamma^*_{\text{RBF}}, \gamma^*_{\text{Lap}}, \lambda^*)$ where Cauchy's infinite spectral variance ($\sigma_p^2 = +\infty$) penalizes Fourier waves.

For the **RBF family** (Figure 4A–B), Gram error confirms ORF < Cos-Sin Pair < Phase Cosine at identical $D^{-1/2}$ slopes. Downstream SVM test-error gains, however, are modest (1–2.6 percentage points at $D = 50$, converging by $D = 1{,}000$), and at $D = 10$ Cos-Sin Pair performs *worse* (22.9% vs. 15.1% on `Synthetic_d2`) because $D/2 = 5$ pairs sample half as many wave directions. For the **Laplacian family** (Figure 4C–D), heavy-tailed Cauchy outliers degrade Cauchy RFF, so Random Binning achieves 2.5–2.8× lower Gram error at $P = 200$ and

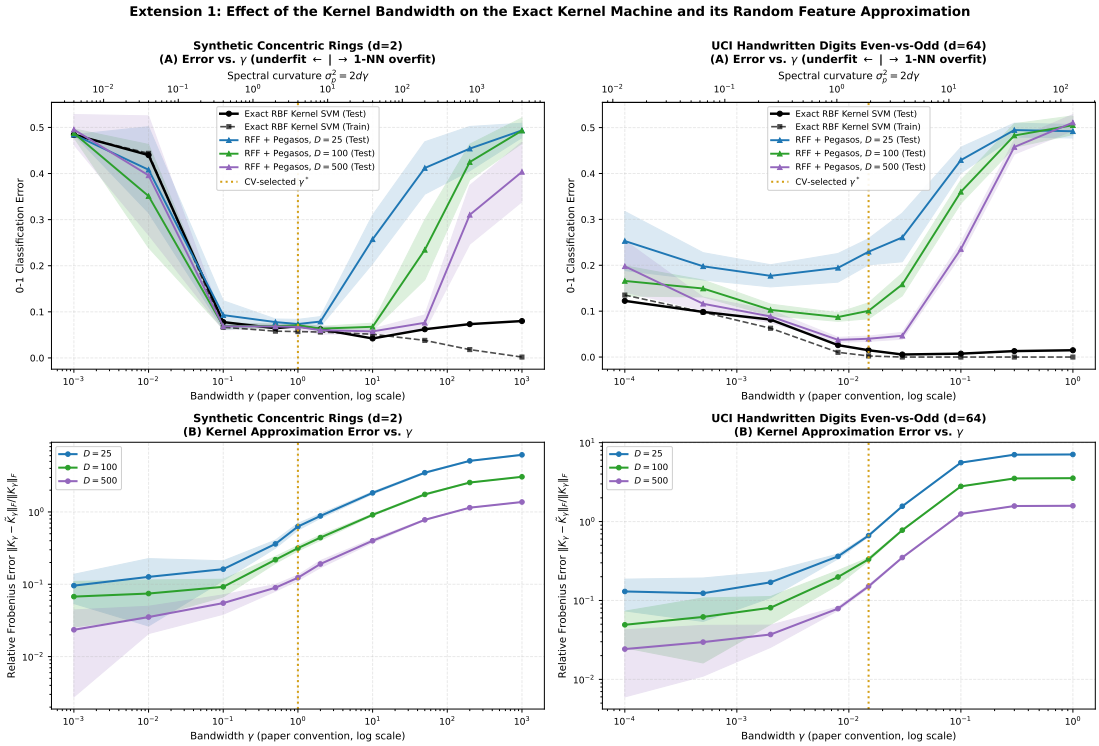


Figure 3: Extension 1 ($S = 10$ seeds, mean ±1 std; top axis: $\sigma_p^2 = 2d\gamma$). **(A)** Error vs. γ for Exact RBF and RFF ($D \in \{25, 100, 500\}$). **(B)** Relative Frobenius error $\mathcal{E}_{\text{Frob}}(\gamma, D)$.

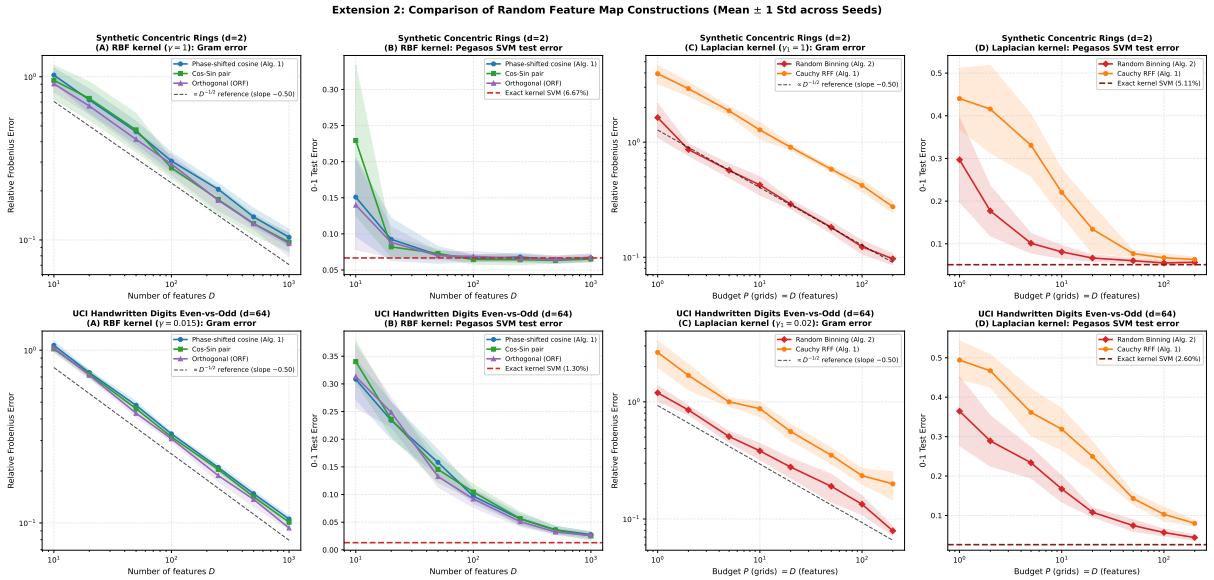


Figure 4: Extension 2 ($S = 10$ seeds, mean ±1 std). **(A–B)** RBF Gram error and SVM test error. **(C–D)** Laplacian Gram error and SVM test error for Random Binning (P grids) vs. Cauchy RFF ($D = P$).

markedly lower SVM test error (4.40% vs. 8.01% on `Digits_d64`, Exact Laplacian 2.60%). Negatively, matching P grids to $D = P$ equates non-zero evaluations per sample rather than parameter count: at $P = 200$, Random Binning expands total sparse dimension D_{tot} beyond 4,000 weights (> 20× Cauchy's features) and maps unseen high-dimensional test cells to zero.

5.3 Extension 3: Runtime, Memory, and Sample-Size Scaling

Under single-thread ($S = 3$ runs, $T = 10{,}000$), performances of `RFFCosinePhase + PrimalPegasosSVM` against `Exact RBF KernelPegasosSVM` are tested in two regimes: **Part A** varies $D \in [5, 2{,}000]$ at fixed training size ($m_{\text{train}} = 1{,}050$ and $1{,}258$), expecting a training-time crossover near $D \approx m_{\text{train}}$; **Part B** fixes $D = 300$ on `Synthetic_d2` ($m_{\text{test}} = 1{,}000$) and scales $m_{\text{train}} \in [250, 6{,}000]$ to test $O(m_{\text{train}})$ vs. $O(m_{\text{train}}^2)$ training and $O(1)$

vs. $O(m_{\text{train}})$ inference.

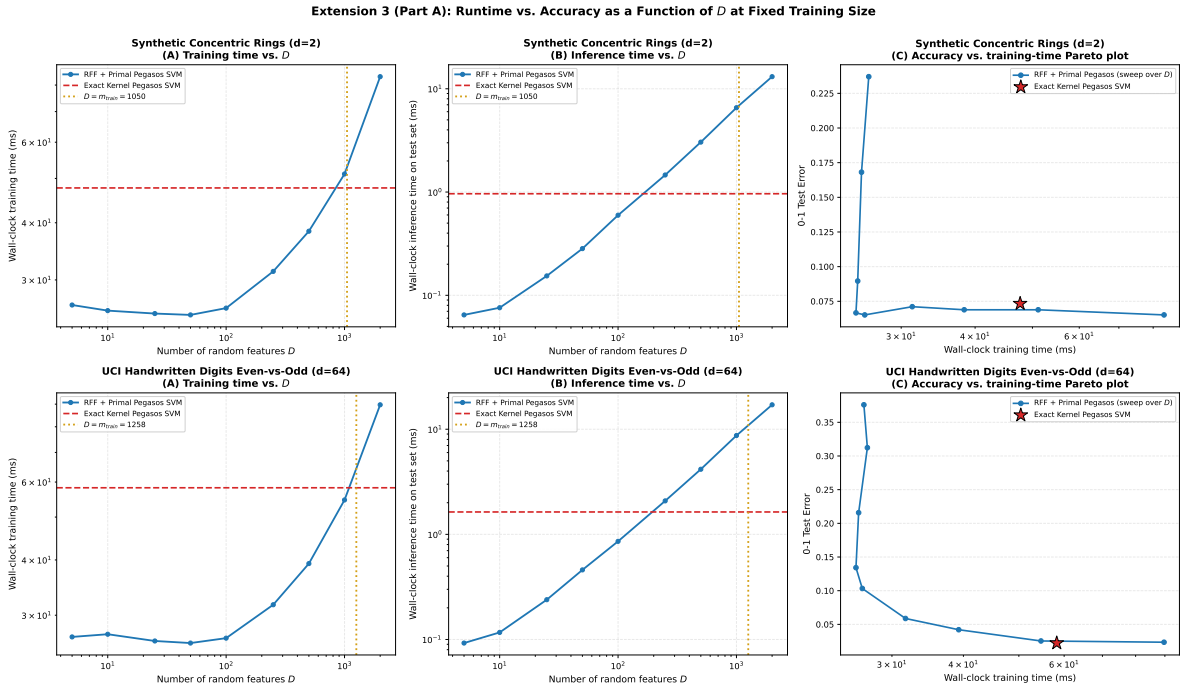


Figure 5: Extension 3, Part A ($S = 3$, single-thread BLAS, $T = 10{,}000$): **(A)** training time vs. D, **(B)** test inference time vs. D, and **(C)** accuracy vs. training-time Pareto frontier.

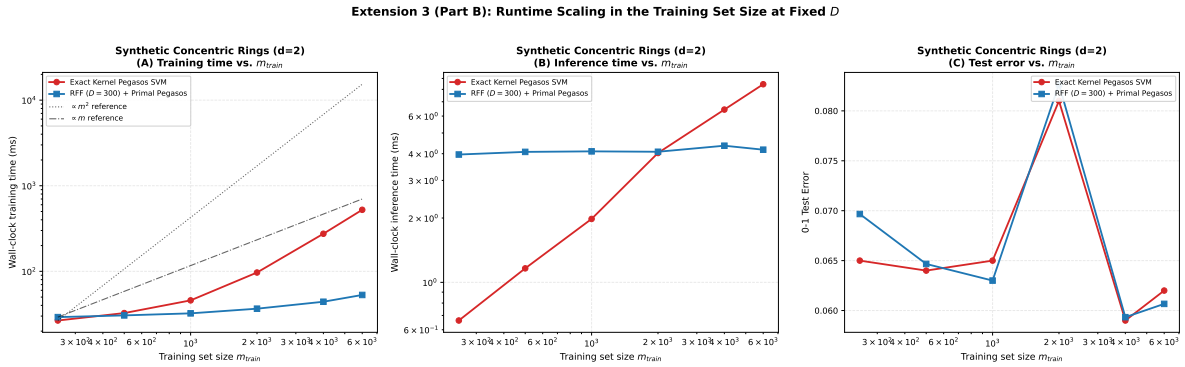


Figure 6: Extension 3, Part B on `Synthetic_d2` at fixed $D = 300$ ($m_{\text{test}} = 1{,}000$): **(A)** training time, **(B)** inference time, and **(C)** test error vs. m_{train}.

In **Part A** (Figure 5), RFF training beats Exact Kernel up to $D \approx 1{,}000 \approx m_{\text{train}}$ before becoming slower at $D = 2{,}000$. Negatively, on small datasets ($m_{\text{train}} \approx 10^3$), RFF inference crosses Exact Kernel much earlier ($D \approx 150$–200) because `np.cos` evaluations carry a higher per-entry constant than matrix multiplication.

Part B (Figure 6) shows how the large-sample regime $D \ll m_{\text{train}}$ reverses this trade-off: as m_{train} grows 24×, Exact Kernel training scales quadratically (26.6 → 523.5 ms, 19.7×) while RFF ($D = 300$) grows only 1.8× (29.2 → 53.0 ms, **9.88**× speedup). Likewise, Exact Kernel query latency (0.66 → 8.45 ms) and storage (18,000 floats) grow linearly with support vectors, whereas RFF remains $O(1)$ invariant (∼ 4 ms inference, 1,200 floats, 15× smaller) within ≤ 0.5 percentage points of Exact Kernel test error (6.07% vs. 6.20% at $m_{\text{train}} = 6{,}000$).

6 Conclusions

Rahimi and Recht's paper shows that shift-invariant kernels can be approximated explicitly through a randomized map $\mathbf{z} : \mathbb{R}^d \to \mathbb{R}^D$, turning non-parametric kernel machines back into standard linear predictors. Experiments confirmed the theoretical $O(D^{-1/2})$ convergence rate and showed the main practical advantage of this idea when $D \ll m_{\text{train}}$: at $m_{\text{train}} = 6{,}000$ with $D = 300$, RFF matched the exact RBF SVM within 0.5% test error while training almost 10× faster with constant memory and query time.

Other noteworthy considerations:

- On `Digits_d64` ($d = 64$, $m_{\text{train}} = 1{,}258$), setting $D < d$ gave worse accuracy than a basic linear SVM, while approaching exact-kernel accuracy required $D \approx 1{,}000 \approx m_{\text{train}}$, removing the speed benefit;
- Furthermore, RFF degraded sharply to chance level at large bandwidths γ due to frequency aliasing, and Random Binning outperformed Cauchy RFF on the Laplacian kernel at the cost of much larger sparse feature vectors.